GST Router API Documentation

This document describes every API exposed by [`routes/gst_router.py`](/abs/path/does/not/exist) in the cu
rrent codebase.

Base router:

- Prefix: `/v1/gst`
- Tag: `gst`
- Source: [routes/gst_router.py](D:/projects/underwritting_tool_backend/routes/gst_router.py:1)
- Controller implementation: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritt
ing_tool_backend/controller/gst_contoller/gst_analyser_controller.py:1)

Overview

The GST module supports five main business capabilities:

1. Read the logged-in user's saved GSTIN.
2. Update the logged-in user's GSTIN.
3. Fetch GST basic profile information for a GSTIN.
4. Complete the OTP-based GST authentication flow required by ScoreMe.
5. Trigger GST report generation and track its async completion using reference IDs.

Authentication

All `/v1/gst/*` APIs are protected by the authorization middleware.

- Middleware source: [middleware/authorization_middleware.py](D:/projects/underwritting_tool_backend/midd
leware/authorization_middleware.py:1)
- Auth mechanism: cookie-based JWT
- Required cookie: `access_token`
- If the cookie is missing: `401 Unauthorized` with body:

--- json ---
{
  "message": "Unauthorized Access"
}

- If the token is invalid: `401 Unauthorized` with body:

--- json ---
{
  "message": "Invalid Token"
}

The middleware extracts:

- `request.state.user_id`
- `request.state.role`

Most GST APIs depend on `request.state.user_id` to read or write user-linked records.

External Dependencies

The GST controllers integrate with ScoreMe GST sandbox APIs configured in [config/config.py](D:/projects/
underwritting_tool_backend/config/config.py:1):

- `SCOREME_GST_INFO_URL`: `/gst/external/gstinbasicinfo`
- `SCORME_GST_USER_NAME_OTP`: `/gst/external/gstgenerateotp`
- `SCOREME_GST_OTP_AUTHENTICATION`: `/gst/external/gstauthentication`
- `SCOREME_GST_POST_GSTIN`: `/gst/external/postgstreport`

Required outbound headers for ScoreMe calls:

- `clientId`: from env `CLIENT_ID`
- `clientSecret`: from env `CLIENT_SECRET`

MongoDB Collections Used

The GST flow writes to multiple collections:

- `users`: stores the logged-in user profile and `gst_number`
- `gstin_basic_info`: cached GST basic information by GSTIN
- `gst_otp_manager`: internal OTP lifecycle tracking
- `gst_reference`: async report request tracking by reference ID
- `gst_analyzed_report`: final saved GST analysis report from webhook processing

API Summary

Method	Path	Purpose
`GET`	`/v1/gst/gstin`	Get the logged-in user's saved GSTIN
`PATCH`	`/v1/gst/gstin`	Save or update the logged-in user's GSTIN
`POST`	`/v1/gst/gstin-basic-info`	Fetch GST basic information for a GSTIN
`POST`	`/v1/gst/generate-otp`	Generate GST OTP for GST authentication
`POST`	`/v1/gst/validate-otp`	Validate generated GST OTP
`POST`	`/v1/gst/post-gstin`	Submit GST report generation request
`POST`	`/v1/gst/get-gst-ref-status`	Fetch status for one or more GST reference IDs
`GET`	`/v1/gst/users-ref-ids`	Fetch all GST reference IDs created by the logged-in user

---

1. Get Saved GSTIN

- Method: `GET`
- Path: `/v1/gst/gstin`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/controller/gst_contoller/gst_analyser_controller.py:25)

Purpose

Returns whether the logged-in user already has a GST number saved in the `users` collection.

Request

No request body.

Processing Workflow

1. Read `request.state.user_id` from auth middleware.
2. Query `users` collection using `_id = ObjectId(user_id)`.
3. If user is not found, return `404`.
4. If `gst_number` exists, return it with `is_found = true`.
5. Otherwise return `gst_number = null` with `is_found = false`.

Success Responses

GST number found:

--- json ---

```
{
  "message": "GST number found",
  "gst_number": "27ABCDE1234F1Z5",
  "is_found": true
}
```

GST number not found:

--- json ---

```
{
  "message": "GST number not found",
  "gst_number": null,
```

```
  "is_found": false
}
```

Error Responses

- `404`

```
--- json ---
{
  "detail": {
    "message": "user not found"
  }
}
```

- `500`

```
--- json ---
{
  "detail": {
    "message": "Internal server error contact the administrator"
  }
}
```

Notes

- This endpoint only reads from `users`.
- It does not validate GSTIN structure because it assumes stored data is already trusted.

---
2. Update Saved GSTIN

- Method: `PATCH`
- Path: `/v1/gst/gstin`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/controller/gst_contoller/gst_analyser_controller.py:69)

Purpose

Stores or updates the logged-in user's GSTIN in the `users` collection.
Request Body

```
--- json ---
{
  "gstin": "27ABCDE1234F1Z5"
}
```

Validation Rules

- Body must be valid JSON.
- Body must not be empty.
- `gstin` is required.
- `gstin` must match this regex:

```
--- text ---
^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$
```

The controller uppercases the supplied GSTIN before validation and storage.
Processing Workflow

1. Parse request JSON.
2. Validate presence of `gstin`.

3. Validate GSTIN format.
4. Verify that the authenticated user exists in `users`.
5. Update `users.gst_number` and `updated_at`.
6. Return the normalized uppercase GSTIN.
Success Response

--- json ---
{
  "message": "GSTIN updated successfully",
  "data": {
    "gstin": "27ABCDE1234F1Z5"
  }
}

Error Responses

- `400` invalid JSON

--- json ---
{
  "detail": {
    "message": "Invalid JSON format in request body"
  }
}

- `400` empty body

--- json ---
{
  "detail": {
    "message": "Body cannot be empty"
  }
}

- `400` missing GSTIN

--- json ---
{
  "detail": {
    "message": "GSTIN is required"
  }
}

- `422` invalid GSTIN format

--- json ---
{
  "detail": {
    "message": "Invalid GSTIN format"
  }
}

- `404` user not found

--- json ---
{
  "detail": {
    "message": "User not found"

```
  }
}
```

Notes

- The code treats `modified_count == 0` as failure. That means sending the same GSTIN value already store
d may return `500` even though the value is effectively already present.

---
3. Fetch GST Basic Info

- Method: `POST`
- Path: `/v1/gst/gstin-basic-info`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/co
ntroller/gst_contoller/gst_analyser_controller.py:122)
Purpose

Fetches basic GST business information for a GSTIN. The controller first checks local cache in MongoDB an
d only calls ScoreMe if no cached record exists.
Request Body

--- json ---
```
{
  "gstin": "27ABCDE1234F1Z5"
}
```

Returned Fields

- `gstin`
- `legalNameOfBusiness`
- `tradeName`
- `gstinStatus`
- `taxpayerType`
- `constitutionOfBusiness`
- `natureOfBusiness`
- `dateOfRegistration`
Processing Workflow

1. Parse request JSON.
2. Validate body and `gstin`.
3. Search `gstin_basic_info` collection by uppercase GSTIN.
4. If found, return cached data.
5. If not found:
   - Call ScoreMe `gstinbasicinfo`.
   - Map known ScoreMe error codes to API errors.
   - Save returned `data` into `gstin_basic_info` with `user_id`.
   - Return a filtered response payload.
Success Response

--- json ---
```
{
  "message": "GST information fetched successfully",
  "data": {
    "gstin": "27ABCDE1234F1Z5",
    "legalNameOfBusiness": "ABC PRIVATE LIMITED",
    "tradeName": "ABC",
    "gstinStatus": "Active",
    "taxpayerType": "Regular",
    "constitutionOfBusiness": "Private Limited Company",
    "natureOfBusiness": ["Retail Business"],
    "dateOfRegistration": "2019-04-01"
  }
```

}

Error Responses

- `400` invalid JSON
- `400` empty body
- `400` missing GSTIN
- ScoreMe-mapped errors from `SCOREME_GST_BASIC_INFO_ERROR_MAP`:
  - `400`: `EBF017`, `EIP018`, `EPI022`
  - `404`: `EGI404`
  - `409`: `EDP1209`
  - `429`: `ERE1203`
  - `500`: `EUP007`
- `500` generic internal errors

Example mapped response:

--- json ---

{
  "detail": {
    "message": "GSTIN Not found Or Incorrect GSTIN.",
    "responseCode": "EGI404"
  }
}

Notes

- This endpoint acts as a cache-backed read-through service.
- The cached document includes `user_id`, but cache lookup is by `gstin` only.

---
4. Generate GST OTP

- Method: `POST`
- Path: `/v1/gst/generate-otp`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/controller/gst_contoller/gst_analyser_controller.py:218)

Purpose

Initiates the GST authentication flow by asking ScoreMe to generate an OTP for the GSTIN and username combination.

Request Body

--- json ---

{
  "gstin": "27ABCDE1234F1Z5",
  "user_name": "registered-gst-username"
}

Processing Workflow

1. Parse request JSON.
2. Validate body, `gstin`, and `user_name`.
3. Call ScoreMe `gstgenerateotp`.
4. Map known ScoreMe errors.
5. On ScoreMe success code `SRO037`:
   - Generate internal `otp_reference_id` as UUID.
   - Insert a tracking document into `gst_otp_manager`.
   - Mark:
     - `is_authenticated = false`
     - `is_expired = false`
     - `otp_generated_at = now`

6. Return the GSTIN and internal `otp_reference_id`.
Success Response

--- json ---
{
  "message": "OTP Generated Successfully",
  "data": {
    "gstin": "27ABCDE1234F1Z5",
    "otp_reference_id": "7d7ed196-6fca-4ac7-8198-70f4f7784d31"
  }
}

Error Responses

- `400` invalid JSON
- `400` empty body
- `400` missing required fields

--- json ---
{
  "detail": {
    "message": "Both gstin and user_name is required"
  }
}

- ScoreMe-mapped errors from `SCOREME_GST_OTP_ERROR_MAP`:
  - `400`: `EBF017`, `EIP018`, `EPI022`, `EGU036`, `ENG034`
  - `401`: `EAA049`
  - `429`: `EAS517`
  - `500`: `ERO038`
  - `503`: `EIS050`

Example mapped response:

--- json ---
{
  "detail": {
    "message": "OTP already sent. Please try again later.",
    "responseCode": "EAS517"
  }
}

Notes

- `otp_reference_id` is internal to this backend. It is not the ScoreMe reference ID.
- This ID must be supplied to `/v1/gst/validate-otp`.

---
5. Validate GST OTP

- Method: `POST`
- Path: `/v1/gst/validate-otp`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/controller/gst_contoller/gst_analyser_controller.py:291)
Purpose

Validates the OTP entered by the user and finalizes GST authentication for that generated OTP reference.
Request Body

--- json ---
{

```
  "gstin": "27ABCDE1234F1Z5",
  "otp": "123456",
  "otp_reference_id": "7d7ed196-6fca-4ac7-8198-70f4f7784d31"
}
```

Processing Workflow

1. Parse request JSON.
2. Validate presence of `gstin`, `otp`, and `otp_reference_id`.
3. Look up `otp_reference_id` in `gst_otp_manager`.
4. Reject if:
   - no tracking document exists
   - OTP already authenticated
   - OTP already expired
5. Call ScoreMe `gstauthentication`.
6. Map known ScoreMe errors.
7. On ScoreMe success code `SAC041`:
   - Update `gst_otp_manager`
   - Set `is_authenticated = true`
   - Set `is_expired = true`
   - Set `authenticated_at = now`
8. Return validation success.

Success Response

--- json ---

```
{
  "message": "OTP Validated Successfully",
  "data": {
    "gstin": "27ABCDE1234F1Z5",
    "is_otp_validated": true
  }
}
```

Error Responses

- `400` invalid JSON
- `400` empty body
- `400` missing required fields

--- json ---

```
{
  "detail": {
    "message": "gstin, otp and otp_reference_id are required"
  }
}
```

- `404` unknown `otp_reference_id`

--- json ---

```
{
  "detail": {
    "message": "Invalid otp_reference_id. No OTP request found."
  }
}
```

- `400` OTP already used

--- json ---

```
{
  "detail": {
```

```
    "message": "OTP already authenticated for this reference ID."
  }
}
```

- `400` OTP expired

```
--- json ---
{
  "detail": {
    "message": "OTP has expired. Please regenerate OTP."
  }
}
```

- ScoreMe-mapped errors intended by `SCOREME_GST_OTP_VALIDATE_ERROR_MAP`:
  - `400`: `EBF017`, `EIP018`, `EPI022`, `EGO040`, `ENG034`, `EGO039`, `EGU051`, `EGO045`
  - `401`: `EAU043`
  - `503`: `EIS042`

Important Implementation Note

The current `SCOREME_GST_OTP_VALIDATE_ERROR_MAP` stores tuple values, but `raise_gst_validate_otp_excepti
on()` reads them as dictionaries. In the current code, any mapped validation error can trigger an interna
l exception path instead of the intended clean HTTP response. The success flow is implemented correctly,
but error mapping for this endpoint is currently inconsistent.

---

6. Submit GST Report Request

- Method: `POST`
- Path: `/v1/gst/post-gstin`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/co
ntroller/gst_contoller/gst_analyser_controller.py:406)

Purpose

Submits a GST report generation request to ScoreMe for a GSTIN and month range. This is an async process.
 The API returns immediately with a GST reference ID, and the final result arrives later through webhook
processing.

Request Body

```
--- json ---
{
  "gstin": "27ABCDE1234F1Z5",
  "from_month": "2024-01",
  "to_month": "2024-12"
}
```

Preconditions

This request assumes GST OTP generation and authentication have already been completed successfully for t
he GSTIN. If not, ScoreMe can reject the request.

Processing Workflow

1. Parse request JSON.
2. Validate body and required fields:
   - `gstin`
   - `from_month`
   - `to_month`
3. Call ScoreMe `postgstreport`.
4. Map known ScoreMe error codes.
5. If ScoreMe success code is `SRS016`:
   - Insert a request-tracking record into `gst_reference`.

  - Store:
    - `user_id`
    - `reference_id`
    - original input
    - month range
    - `gst_reference_id_status = INPROGRESS`
    - request metadata
    - webhook placeholders
6. Return `202 Accepted` with the generated `gst_reference_id`.
Success Response

--- json ---
{
  "message": "gstin sent successfully",
  "data": {
    "gstin": "27ABCDE1234F1Z5",
    "gst_reference_id": "SMGSTREF123456"
  }
}

Database Record Created

The `gst_reference` document includes:

- `gstin`
- `user_id`
- `reference_id`
- `input_data`
- `from_month`
- `to_month`
- `gst_reference_id_status = "INPROGRESS"`
- `gst_request_status`
- `gst_request_response_code`
- `gst_request_initiated_time`
- `webhook_status = "PENDING"`
- `webhook_received_time = null`
- `webhook_response_code = null`
- `webhook_reesponse_message = null`
- `gst_report_url = null`
- `is_consumed = false`
- `consumed_at = null`
Error Responses

- `400` invalid JSON
- `400` empty body
- `400` missing required fields

--- json ---
{
  "detail": {
    "message": "gstin and from_month and to_month are required"
  }
}

- `404`

--- json ---
{
  "detail": {
    "message": "Gst data not found for this months"
  }

}

- ScoreMe-mapped errors from `SCOREME_GST_POST_GSTIN_ERROR_MAP`:
  - `400`: `EBF017`, `EIP018`, `EPI022`, `EVP085`, `ENG034`, `EPG044`, `ERT146`
  - `401`: `EOA048`, `EAE052`
  - `409`: `EDP1209`
  - `429`: `ERE1203`
  - `503`: `EIS042`

Example mapped response:

--- json ---
{
  "detail": {
    "message": "Please complete the GST OTP generation and authentication process.",
    "responseCode": "EOA048"
  }
}

Notes

- This endpoint does not return the final GST report.
- The response only confirms that report generation has started.

---
7. Get GST Reference Status

- Method: `POST`
- Path: `/v1/gst/get-gst-ref-status`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/controller/gst_contoller/gst_analyser_controller.py:490)
Purpose

Returns the current internal status for one or more GST reference IDs previously created by `/v1/gst/post-gstin`.
Request Body

--- json ---
{
  "gst_ref_id": [
    "SMGSTREF123456",
    "SMGSTREF123457"
  ]
}

Processing Workflow

1. Parse request JSON.
2. Validate non-empty body.
3. Validate `gst_ref_id`.
4. For each input reference ID:
   - Search `gst_reference` by `reference_id`
   - Read `gst_reference_id_status`
5. Return only the reference IDs found in DB.
Success Response

--- json ---
{
  "message": "status fetch successfully",
  "data": {
    "gst_reference_id_status": [

```
      {
        "gst_reference_id_status": "INPROGRESS",
        "gst_reference_id": "SMGSTREF123456"
      },
      {
        "gst_reference_id_status": "COMPLETED",
        "gst_reference_id": "SMGSTREF123457"
      }
    ]
  }
}
```

Error Responses

- `400` invalid JSON
- `400` empty body
- `400` missing reference ids

```
--- json ---
{
  "detail": {
    "message": "At least one gst reference id is required"
  }
}
```

- `500` generic internal error

Notes

- Unknown reference IDs are silently omitted from the result.
- This endpoint does not enforce ownership; it only checks whether a reference ID exists in `gst_referenc
e`.

---

8. Get All GST Reference IDs for Logged-In User

- Method: `GET`
- Path: `/v1/gst/users-ref-ids`
- Source: [controller/gst_contoller/gst_analyser_controller.py](D:/projects/underwritting_tool_backend/co
ntroller/gst_contoller/gst_analyser_controller.py:525)

Purpose

Fetches all GST report request references created by the authenticated user.

Processing Workflow

1. Read `request.state.user_id`.
2. Query `gst_reference` for all records with that `user_id`.
3. Return a reduced projection:
   - `gst_reference_id_status`
   - `from_month`
   - `to_month`
   - `reference_id`

Success Response

```
--- json ---
{
  "message": "Gst reference id list fetch success",
  "data": [
    {
      "gst_reference_id_status": "INPROGRESS",
      "from_month": "2024-01",
      "to_month": "2024-12",
```

```
      "reference_id": "SMGSTREF123456"
    },
    {
      "gst_reference_id_status": "COMPLETED",
      "from_month": "2023-01",
      "to_month": "2023-12",
      "reference_id": "SMGSTREF123400"
    }
  ]
}
```

Error Responses

- `404`


```
--- json ---
{
  "detail": {
    "message": "No reference id found for this user"
  }
}
```

- `500` generic internal error

Notes

- This is the user-specific listing endpoint and is safer than `/get-gst-ref-status` for building a "my r equests" UI.

End-to-End GST Workflow

This is the expected business flow for a client application consuming the GST module.

Phase 1: Save GSTIN

1. Call `GET /v1/gst/gstin` to check whether a GSTIN already exists.
2. If missing or outdated, call `PATCH /v1/gst/gstin`.

Phase 2: Fetch GST Business Profile

1. Call `POST /v1/gst/gstin-basic-info` with the GSTIN.
2. Use returned business data to confirm the GSTIN belongs to the correct entity.

Phase 3: Authenticate GST Access

1. Call `POST /v1/gst/generate-otp` with:
   - `gstin`
   - `user_name`
2. Receive `otp_reference_id`.
3. Prompt the user to enter the OTP received through the GST system.
4. Call `POST /v1/gst/validate-otp` with:
   - `gstin`
   - `otp`
   - `otp_reference_id`
5. Continue only after successful OTP validation.

Phase 4: Generate GST Report

1. Call `POST /v1/gst/post-gstin` with:
   - `gstin`
   - `from_month`
   - `to_month`
2. Receive `gst_reference_id`.
3. Persist this ID on the client side if needed.

Phase 5: Track Async Processing

1. Poll `POST /v1/gst/get-gst-ref-status` with one or more reference IDs.
2. Or list all requests for the logged-in user with `GET /v1/gst/users-ref-ids`.
3. Watch for status changes such as:
   - `INPROGRESS`
   - `COMPLETED`
   - `FAILED`

Phase 6: Webhook Completion

When ScoreMe finishes report generation, it calls the webhook endpoint:

- Route: [routes/webhook_router.py](D:/projects/underwritting_tool_backend/routes/webhook_router.py:1)
- Path: `/webhook/gst-statements`

Webhook flow in the current code:

1. Raw webhook payload is stored in `gststatements`.
2. If `responseCode != "SRC001"`:
   - mark `gst_reference.gst_reference_id_status = FAILED`
   - mark webhook metadata as received
3. If `responseCode == "SRC001"`:
   - mark `gst_reference.gst_reference_id_status = COMPLETED`
   - store webhook metadata
   - fetch `jsonUrl` from ScoreMe
   - store parsed report in `gst_analyzed_report`

Sequence Summary

--- text ---
Client
  -> GET /v1/gst/gstin
  -> PATCH /v1/gst/gstin
  -> POST /v1/gst/gstin-basic-info
  -> POST /v1/gst/generate-otp
  -> POST /v1/gst/validate-otp
  -> POST /v1/gst/post-gstin
  -> receives gst_reference_id
  -> POST /v1/gst/get-gst-ref-status (polling)

ScoreMe
  -> POST /webhook/gst-statements

Backend
  -> update gst_reference
  -> fetch final JSON report
  -> save gst_analyzed_report

State Transitions
OTP State

`gst_otp_manager` lifecycle:

1. After `/generate-otp`
   - `is_authenticated = false`
   - `is_expired = false`
2. After successful `/validate-otp`
   - `is_authenticated = true`
   - `is_expired = true`

GST Report Request State

`gst_reference.gst_reference_id_status` lifecycle:

1. Immediately after `/post-gstin`: `INPROGRESS`
2. After successful webhook and report save: `COMPLETED`

3. After failure webhook response: `FAILED`

Known Implementation Gaps

These are relevant if this documentation is also being used as implementation reference.

1. `raise_gst_validate_otp_exception()` expects dictionary values, but `SCOREME_GST_OTP_VALIDATE_ERROR_MAP` currently uses tuples.
2. `/v1/gst/get-gst-ref-status` does not verify that the requested reference IDs belong to the authenticated user.
3. Updating GSTIN with the same value may return a failure because `modified_count == 0` is treated as an error.
4. The webhook route currently reads `request.app.state.user_id`, but user identity is normally stored on `request.state.user_id` by middleware. The webhook path is public, so this value may not exist during runtime.
5. `controller/webhook/scoreme_webhook_controller.py` calls `gst_webhook_consumer(input_data=...)`, while the consumer function signature expects `webhook_data=...`.

Suggested Client Workflow

For frontend or API consumer integration, the safest order is:

1. `GET /v1/gst/gstin`
2. `PATCH /v1/gst/gstin` if needed
3. `POST /v1/gst/gstin-basic-info`
4. `POST /v1/gst/generate-otp`
5. `POST /v1/gst/validate-otp`
6. `POST /v1/gst/post-gstin`
7. `GET /v1/gst/users-ref-ids` or `POST /v1/gst/get-gst-ref-status`

This order matches the controller logic and ScoreMe dependency chain.